

Aula 6 — Modulação por largura de pulso

Um pino de dois níveis entregando qualquer fração da potência

Microcontroladores — DENE/UFMT

OBJETIVOS

- Justificar quando a razão cíclica equivale a uma tensão média, pela constante de tempo da carga.
- Descrever o mecanismo do CCP em modo PWM: Timer2, PR2, o comparador de dez bits e a cópia sincronizada da razão cíclica.
- Derivar período e resolução a partir de PR2 e do divisor, e escolher entre os dois para uma aplicação dada.
- Configurar o módulo na ordem da folha de dados e escrever a razão cíclica nos dois registradores que a guardam.
- Reconhecer os limites do módulo: frequência mínima, os 100% que não cabem, um Timer2 para dois canais.
- Dimensionar um filtro RC que transforma PWM em tensão, pela ondulação e pelo tempo de resposta.

Este encontro tem duas partes. Os primeiros 70 minutos são esta aula. Os últimos 50 são a **avaliação integradora I**, sobre os encontros 0 a 5, e não há miniteste. O PWM desta aula entra no miniteste do encontro 7.

ATENÇÃO

1 Gerar em vez de contar

Na aula 5 o contador recebeu pulsos, e o processador leu o resultado. O terceiro uso inverte o sentido: o Timer2, acoplado a um comparador, **produz** um sinal no pino, sem o processador.

O Timer2 conta até o valor de PR2 e recomeça. Um comparador liga o pino no recomeço e o desliga quando a contagem alcança a razão cíclica escrita pelo programa.

CONCEITO

O resultado é uma onda quadrada de período fixo, dado por PR2, e tempo ligado ajustável. Mudar o tempo ligado é uma escrita num registrador; depois disso, o hardware repete a forma de onda sozinho, para sempre.

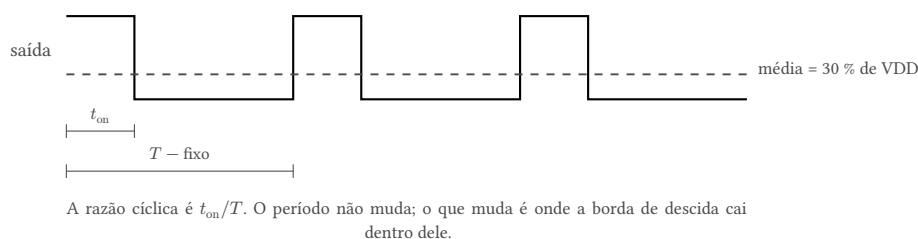


Figura 1: A razão cíclica é a única grandeza que muda. Frequência e amplitude ficam fixas.

Ligar e desligar uma carga rápido o bastante entrega a ela uma fração da energia, sem nenhum componente analógico no caminho.

A razão cíclica só equivale a uma tensão média se a carga for lenta comparada ao período.

CONCEITO

O que integra o sinal não é o microcontrolador: é a carga. A inércia mecânica da ventoinha, a inércia térmica do aquecedor e a persistência da retina fazem o trabalho. Se a carga responder mais rápido que o período, ela não vê uma média — vê exatamente o que existe, um interruptor batendo.

Carga	Constante de tempo	Frequência mínima razoável
LED	olho: ≈ 20 ms	acima de 100 Hz, ou se vê a cintilação
Ventoinha	inércia do rotor: ≈ 100 ms	algumas centenas de Hz bastam
Aquecedor	inércia térmica: dezenas de segundos	qualquer coisa acima de 1 Hz
Relé	contato mecânico: ms, e desgasta	nunca

Relé não faz PWM. Cada comutação é um evento mecânico com vida útil contada, e comutar a 1 kHz destrói o contato em minutos. O aquecedor acionado por relé é uma saída de duas posições — e a oscilação que o R9 vai medir é consequência direta disso. O estágio que permite PWM no aquecedor é o do encontro 8.

ATENÇÃO

2 Dentro do módulo

O PIC18F4550 tem dois módulos de captura, comparação e PWM, CCP1 e CCP2. Em modo PWM, os dois usam o Timer2 como base de tempo.

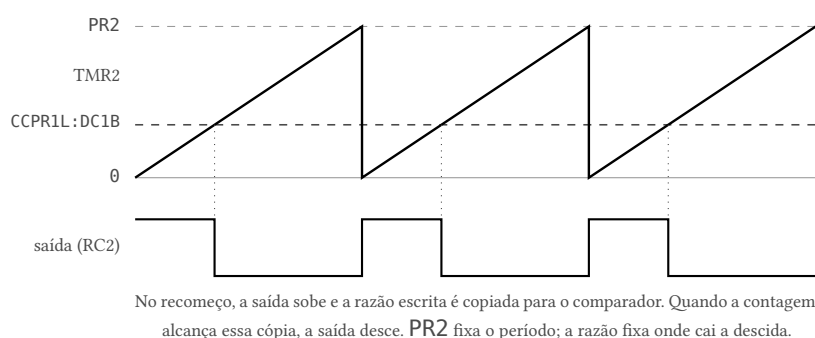


Figura 2: O Timer2 sobe até PR2 e recomeça. A saída sobe no recomeço e desce quando a contagem alcança a razão cíclica.

Em cada período, três coisas acontecem, sempre na mesma ordem:

1. **Recomeço.** Quando TMR2 alcança PR2, o incremento seguinte o zera. No mesmo instante, a saída sobe, e a razão cíclica escrita pelo programa é **copiada** para o registrador que o comparador de fato usa (CCPR1H, mais dois bits internos).
2. **Contagem.** TMR2 sobe, um passo a cada $4T_{osc}$ vezes o divisor.
3. **Comparação.** Quando a contagem alcança a cópia, a saída desce, e fica baixa até o próximo recomeço.

2.1 Dez bits num contador de oito

TMR2 tem oito bits, e a razão cíclica tem dez. Os dois bits que faltam vêm de dentro do divisor: com divisor 1:1, das quatro fases do ciclo de máquina; com divisor maior, do próprio contador do divisor. O comparador enxerga, portanto, um contador de dez bits que anda a cada T_{osc} vezes o divisor.

O período tem $(PR2 + 1)$ passos de TMR2, e cada um se divide em quatro para o comparador. A razão cíclica é contada nesses passos menores:

CONCEITO

$$T_{PWM} = (PR2 + 1) \cdot 4 \cdot T_{osc} \cdot \text{divisor}$$

$$t_{on} = \text{razão} \cdot T_{osc} \cdot \text{divisor}$$

$$\text{resolução} = \log_2(4 \cdot (PR2 + 1)) \text{ bits}$$

2.2 A cópia que evita o pulso torto

O programa escreve a razão cíclica quando quer, em qualquer ponto do período. Se o comparador usasse o valor na hora, uma escrita no meio do período poderia cortar o pulso ao meio, ou esticá-lo até o fim, e a carga veria um pulso que ninguém pediu.

A cópia no recomeço resolve isso. O valor novo só vale **a partir do próximo período**, sempre inteiro. É por isso que CCPR1H é só leitura em modo PWM: quem escreve nele é o hardware.

Compare com o PWM feito por software no O1, em que o programa invertia o pino. Ali, mudar a razão no meio do período produzia exatamente o pulso torto — e o período inteiro dependia de o laço chegar a tempo. O módulo tira as duas coisas do programa.

NOTA

3 Período e resolução saem do mesmo registrador

A 16 MHz, $4T_{osc} = 250$ ns. Com PR2 = 255 e divisor 16:

$$T_{PWM} = 256 \cdot 250 \text{ ns} \cdot 16 = 1024 \mu\text{s} \rightarrow 976,6 \text{ Hz, com } \log_2 1024 = 10 \text{ bits}$$

PR2	Divisor	Período	Frequência	Resolução
255	1:16	1024 μs	976,6 Hz	10 bits
255	1:4	256 μs	3,906 kHz	10 bits
255	1:1	64 μs	15,63 kHz	10 bits
199	1:4	200 μs	5,000 kHz	9,6 bits
199	1:1	50 μs	20,00 kHz	9,6 bits
99	1:1	25 μs	40,00 kHz	8,6 bits
63	1:1	16 μs	62,50 kHz	8 bits
15	1:1	4 μs	250,0 kHz	6 bits

Período e resolução são o mesmo botão. Aumentar a frequência exige diminuir PR2, e diminuir PR2 corta bits de razão cíclica. Com divisor 1:1, a conta fecha numa linha só:

CONCEITO

$$\text{resolução máxima} = \log_2(F_{osc}/f_{PWM}) \text{ bits}$$

Cada vez que a frequência dobra, perde-se um bit. O divisor não escapa da regra: ele só baixa a frequência **mantendo** os bits, e nunca a sobe.

A linha de 20 kHz não está na tabela por acaso. É a menor frequência que o ouvido não alcança, e é onde se põe uma ventoinha que não pode apitar. Custa 0,4 bit em relação ao máximo — um preço que quase sempre vale pagar.

NOTA

3.1 Onde moram os dez bits

Os oito bits altos da razão cíclica ficam em CCPR1L. Os dois baixos, r_1 e r_0 , ficam em CCP1CON, misturados com os bits que selecionam o modo:

bit	7	6	5	4	3	2	1	0
CCP1CON	P1M1	P1M0	DC1B1	DC1B0	CCP1M3	CCP1M2	CCP1M1	CCP1M0
PWM simples	0	0	r_1	r_0	1	1	0	0

No PIC18F4550, o CCP1 é o módulo **aprimorado**: P1M1:P1M0 escolhem entre saída simples, meia ponte e ponte completa. Com 00, a saída é só P1A, em RC2. O CCP2 é o módulo comum, e sai em RC1 ou RB3, conforme o bit de configuração CCP2MX.

```
/* razao: 0 a 1023. */
void pwm_razao(uint16_t razao)
{
    if (razao > 1023u) {
        razao = 1023u;
    }
    CCPR1L = (uint8_t)(razao >> 2);
    CCP1CON = (uint8_t)((CCP1CON & 0xCF) |
```

/* 8 bits altos */

```
    | (uint8_t)((razao & 0x03u) << 4)); /* 2 bits baixos */
}
```

A máscara 0xCF preserva tudo menos os bits 5 e 4. É a mesma situação da instrução do encontro 2 e do ADCON0 do encontro 4 — uma palavra repartida em campos de significados diferentes.

Com PR2 = 255 e divisor 16 sai uma coincidência conveniente: o valor de dez bits da razão cíclica vale exatamente o tempo ligado **em microssegundos**, porque cada unidade dura $T_{osc} \cdot 16 = 62,5 \text{ ns} \cdot 16 = 1 \mu\text{s}$. Razão 512 é meio período, e é 512 μs .

NOTA

Vale usar isso em aula e vale desconfiar depois: a coincidência morre se alguém mexer no divisor.

4 Configurar na ordem

A folha de dados dá uma sequência, e cada passo tem motivo:

```
void pwm_iniciar(void)
{
    PR2      = 255;          /* 1. periodo */
    CCP1L    = 0;           /* 2. razao inicial: 0 % */
    LATCbits.LATC2 = 0;     /* 3. pino como saida (LAT antes de TRIS) */
    TRISCbits.TRISC2 = 0;
    T2CON    = 0x06;        /* 4. Timer2 ligado, divisor 1:16 */
    CCP1CON  = 0x0C;        /* 5. modo PWM, saida simples, DC1B = 00 */
}
```

Período e razão vêm antes do modo PWM porque, no instante em que CCP1CON passa a 0x0C, o módulo começa a gerar a onda com o que encontrar. Razão inicial em zero garante que o primeiro período não liga a carga por acidente.

RC2 é o pino do CCP1, e nele chegam a ventoinha (CH3-5), o buzzer (CH3-6) e o DAC (CH3-7) — um de cada vez. RC1 é o CCP2, e nele chegam o aquecedor (relé) e a lâmpada. O CCP2MX pertence ao bootloader, como todo bit de configuração.

NESTE KIT

5 Os limites do módulo

5.1 Os 100% que não cabem

Com PR2 = 255, o período tem $4 \cdot 256 = 1024$ passos. A razão cíclica tem dez bits, e o maior valor que cabe é 1023. Resultado: **o módulo não chega a 100%**. Sobra um pulso baixo de um passo em cada período — 1 μs a cada 1024 μs com divisor 16.

Com PR2 menor, a razão pode passar do período; nesse caso o comparador nunca alcança a cópia, e a saída fica alta o tempo todo. E razão zero mantém a saída baixa. Os extremos existem, mas não em todas as configurações.

Para a ventoinha, 1 μs em 1024 não muda nada. Para um estágio de potência que precisa de 100% de verdade — uma chave que não pode abrir nunca — é um pulso de comutação por período, com perda e ruído. Quem precisa de 100% escolhe PR2 < 255, ou desliga o módulo e fixa o pino.

NOTA

5.2 A frequência mínima

O maior PR2 com o maior divisor dá o período mais longo:

$$T_{\max} = 256 \cdot 250 \text{ ns} \cdot 16 = 1024 \mu\text{s} \rightarrow f_{\min} = 976,6 \text{ Hz}$$

Abaixo disso, o módulo não vai. Para um aquecedor com estágio eletrônico, 1 kHz é mais que suficiente. Para o buzzer, é um problema: toda nota abaixo de 976 Hz fica fora do alcance — e o encontro 7 resolve isso com um temporizador e uma interrupção.

5.3 Dois canais, um Timer2

CCP1 e CCP2 dividem o Timer2. As duas saídas têm, obrigatoriamente, o **mesmo período**; cada uma tem a sua razão cíclica. Se ventoinha e aquecedor forem acionados por PWM, terão a mesma frequência de comutação — restrição que volta no encontro 8.

5.4 O Timer2 continua um temporizador

Enquanto gera o PWM, o Timer2 ainda levanta TMR2IF a cada recomeço, depois do pós-divisor de 1:1 a 1:16. Com PR2 = 249, divisor 16 e pós-divisor 10, isso é uma marca a cada 10 ms exatos — sem pré-carga, porque PR2 não precisa ser reescrito. O período do PWM vira a base de tempo do programa.

O preço é o acoplamento: mudar PR2 para mudar a frequência do PWM muda também a base de tempo. Quem usar os dois precisa decidir primeiro qual deles manda.

ATENÇÃO

6 Do pulso à tensão: o filtro RC

Quando a carga não é lenta o bastante — a entrada de um conversor, um amplificador, uma referência —, o próprio circuito fornece a lentidão: um resistor e um capacitor.

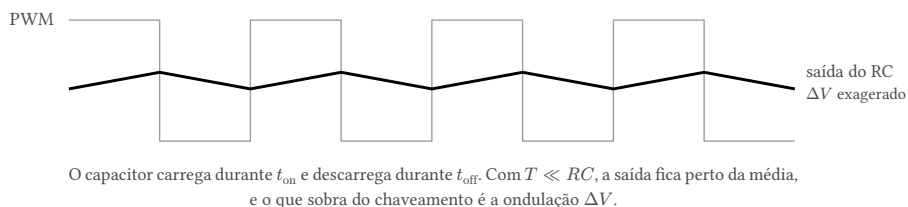


Figura 3: A saída do filtro oscila em torno da média. Quanto maior RC comparado ao período, menor a oscilação.

Com o período muito menor que a constante de tempo ($T \ll RC$), o capacitor carrega quase em linha reta durante t_{on} e descarrega quase em linha reta durante t_{off} . A ondulação de pico a pico fica:

CONCEITO

$$\Delta V \approx V \cdot D(1 - D) \cdot T / (RC)$$

e é máxima em $D = 1/2$, onde vale $VT/(4RC)$.

O preço de uma ondulação pequena é um RC grande, e um RC grande é uma saída lenta: depois de uma mudança de razão, a tensão leva cerca de $5RC$ para chegar ao valor novo.

Exemplo. Quer-se uma tensão de 0 a 5 V com ondulação menor que 1 LSB de um conversor de dez bits (4,9 mV). Qual RC mínimo, e em quanto tempo a saída responde, a 15,6 kHz e a 976 Hz?

TAREFA

A 15,6 kHz, $T = 64 \mu s$:

$$RC \geq \frac{5 \cdot 64 \mu s}{4 \cdot 4,9 \text{ mV}} \approx 16 \text{ ms}, \text{ e } 5RC \approx 82 \text{ ms}$$

A 976 Hz, $T = 1024 \mu s$ — dezesseis vezes mais:

$$RC \geq 262 \text{ ms}, \text{ e } 5RC \approx 1,3 \text{ s}$$

A frequência alta ajuda o filtro. Para a mesma ondulação, o filtro de um PWM dezesseis vezes mais rápido responde dezesseis vezes mais depressa. E a frequência alta custa bits, pela regra da seção anterior.

CONCEITO

Um conversor digital-analógico feito de PWM e RC é, portanto, uma troca de três pontas: resolução, ondulação e velocidade. Escolhem-se duas.

O XM118 tem um filtro desse tipo em RC2, a entrada DAC (CH3-7). Ela fica **desligada o semestre inteiro**, como o O1 estabeleceu. A conta acima vale para qualquer RC, e é ela que interessa.

NESTE KIT

7 A rampa sem espera

O brilho de um LED que sobe e desce é o exemplo clássico de PWM. Feito com `__delay_ms`, ele repete o defeito que a aula 5 condenou: a rampa engasga se o laço tiver outra tarefa.

A marca de 10 ms vem do Timer0, pela pré-carga 0x63C0 da aula 5, e o laço consulta o indicador:

```
#define PRECARGA 25536u /* 65536 - 40000: 10 ms a 250 ns */

void main(void)
{
    int16_t r = 0, passo = 4;

    pwm_iniciar();
    TMR0H = (uint8_t)(PRECARGA >> 8);
    TMR0L = (uint8_t)(PRECARGA & 0xFF);
    T0CON = 0x88; /* ligado, 16 bits, interno, sem divisor */
    INTCONbits.TMR0IF = 0;

    for (;;) {
        if (INTCONbits.TMR0IF) { /* passaram 10 ms */
            INTCONbits.TMR0IF = 0;
            TMR0H = (uint8_t)(PRECARGA >> 8);
            TMR0L = (uint8_t)(PRECARGA & 0xFF);

            r += passo;
            if (r >= 1020) {
                passo = -4;
            } else if (r <= 0) {
                passo = 4;
            }
            pwm_razao((uint16_t)r);
        }
        /* o resto do programa */
    }
}
```

A aula 5 mostrou que consultar o indicador faz o intervalo derivar. Aqui a deriva não importa: um passo de rampa com 12 ms em vez de 10 ms não se vê. Saber **quando** a imprecisão é aceitável é tão parte do projeto quanto saber eliminá-la. Quando não for, a interrupção do encontro 7 resolve.

NOTA

7.1 O olho não é linear

A rampa é linear na razão cíclica, mas não parece linear no brilho: o olho responde muito à variação perto do apagado e quase nada perto do máximo. O brilho dispara no começo da subida e estaciona no fim.

A correção é aplicar à razão uma curva que cresce devagar no começo. Uma parábola já melhora muito, e custa uma multiplicação:

```
/* p: 0 a 255 (brilho percebido); devolve a razao de 0 a 1016 */
uint16_t brilho(uint8_t p)
{
```

```
return (uint16_t)(((uint16_t)p * p) >> 6);
}
```

Verifique o maior produto antes de escolher o tipo, como no R4 e no R5: $255 \cdot 255 = 65025$, que cabe em dezesseis bits por 510. Um brilho de 0 a 256 não caberia.

ATENÇÃO

Circula a afirmação de que a ventoinha “não responde à frequência de comutação” porque a inércia do rotor filtra tudo. A parte mecânica sim; o resto não. O enrolamento sofre magnetostrição na frequência de chaveamento, e uma ventoinha em PWM a 1 kHz apita a 1 kHz, com rotação perfeitamente estável.

DIVERGÊNCIA

A frequência é escolhida também por critério acústico — e a linha de 20 kHz da tabela é a resposta.

8 Três usos, um contador

Uso	Fonte	O que o processador faz
Medir tempo	Relógio interno	Olha o indicador — ou é chamado quando ele sobe
Contar eventos	Pino T0CKI ou T13CKI	Lê o resultado quando quiser
Gerar forma de onda	Relógio interno, via PR2 e CCPR1L	Escreve a razão cíclica e esquece

9 Exercícios

Exercício 6.1. Você precisa de PWM a 5 kHz no cooler, a 16 MHz.

TAREFA

(a) Escolha PR2 e o divisor do Timer2.

(b) Qual resolução de razão cíclica resulta?

(c) Se o projeto exigir 10 bits de resolução, qual é a maior frequência possível?

Exercício 6.2. Explique por que a razão cíclica de 30% aplicada a um LED, a um aquecedor e a um relé produz três resultados de naturezas diferentes.

TAREFA

Exercício 6.3. O programa escreve CCPR1L no meio de um período, com a saída ainda alta.

TAREFA

(a) O período em curso termina com a razão antiga ou com a nova?

(b) Por que isso é uma vantagem, e não um atraso indesejado?

Exercício 6.4. Um colega quer 100% de razão cíclica na ventoinha com PR2 = 255 e escreve `pwm_razao(1023)`.

TAREFA

(a) O que o osciloscópio mostra?

(b) Proponha duas formas de obter 100% de verdade.

Exercício 6.5. Um PWM a 976 Hz, 0 a 5 V, passa por um filtro com $RC = 10$ ms.

TAREFA

(a) Qual a ondulação máxima, e em que razão cíclica ela acontece?

(b) Quantos LSB isso representa para o conversor do encontro 4?

(c) O que você mudaria primeiro: R , C ou a frequência do PWM? Justifique pelo tempo de resposta.

Exercício 6.6. O projeto quer a ventoinha em CCP1 a 25 kHz, silenciosa, e o buzzer em CCP2 tocando notas de 1 a 4 kHz.

TAREFA

(a) É possível? Por quê?

(b) Qual das duas saídas você passaria para outro mecanismo, e qual?

No encontro 7: a interrupção, por inteiro. A função que o hardware chama, GIE e TMR0IE, e o porquê de `volatile` e de `uint8_t` — a dívida da aula 5. Junto vêm o repique

NOTA

que o contador externo revelou, e as notas que o módulo de PWM não alcança: tocar uma melodia no buzzer e escrever na tela ao mesmo tempo.